# Session 3 - How AI Actually Works

**Mentor:** Darsi Gangothri
**Date:** Saturday, 21 Jun 2026
**Module:** 1 - AI/ML and GenAI Foundations
**Course:** LLM Engineering Masterclass

## What We Covered

### 1. The AI Pyramid - AI, ML, DL, LLM

Four layers, each building on the one below:

| Layer | What It Means | Examples |
|---|---|---|
| AI | Any system that mimics human intelligence | Spam filters, thermostats, calculators |
| ML | AI that learns patterns from data instead of hand-written rules | Netflix recommendations, fraud detection |
| DL | ML using neural networks with many layers of pattern recognition | Face unlock, voice assistants, image recognition |
| LLM | Deep learning on text at massive scale (billions of parameters) | ChatGPT, Claude, Llama, Qwen |

As an LLM Engineer, you work at the top of this pyramid.

### 2. How LLMs Work

**Neural Networks (the foundation):**

- A neural network is layers of connected nodes that learn patterns from data
- Each connection has a "weight" that gets adjusted during training
- An LLM is a massive neural network trained on billions of words

**Tokens:**

- LLMs cannot read raw text. They read chunks called tokens
- "The cat sat on the mat" becomes 6 tokens

- "unhappiness" becomes 3 tokens: "un", "happi", "ness"
- Rule of thumb: 100 tokens is roughly 75 words
- Context window = how many tokens the model can see at once (GPT-4: 128K, smaller models: 4K-8K)

**Attention:**

- The model looks at all tokens and decides which ones matter most
- Like skimming a long email and jumping to deadlines and action items
- Self-attention = the model looking at ALL tokens simultaneously to understand relationships

**Next-Token Prediction:**

- An LLM does one thing: predicts the next token
- "The capital of France is ___" predicts "Paris"
- Then takes the full text and predicts the NEXT token, and the next, and the next
- This loop is called autoregressive generation

**The full pipeline:**

```
Text -> Tokens -> Attention -> Predict Next Token -> Output
                                    ^                    |
                                    |____________________|
                                         (loop)
```

## 3. Frontier (Closed) vs Open-Source LLMs

**Closed / Frontier models:**

- GPT-4o, o3 (OpenAI), Claude (Anthropic), Gemini (Google)
- You pay per token
- Your data goes to their servers
- You cannot customize the model

**Open-Source models:**

- Llama 3 (Meta), Mistral (Mistral AI), Qwen 3 (Alibaba), Phi (Microsoft), Gemma (Google), DeepSeek
- Free to download and run
- Data stays on your machine
- You can fine-tune for your domain

**Why open-source matters:**

| Reason | Details |
|---|---|
| Zero cost | No per-token billing. Run it 1000 times for free. |
| Data privacy | Banks, hospitals, government - data never leaves your machine. |
| Customizable | Fine-tune any model to become a domain expert. |
| No vendor lock-in | You own the model. Nobody can change the terms. |

## 4. The GenAI Engineering Lifecycle

The 8 stages of building a GenAI product:

```
Data -> Model Selection -> Prompt Engineering -> RAG -> Fine-tuning -> Age
```

| Stage | What It Means | Real-World Example |
|---|---|---|
| Data | Collecting and preparing information | Zomato restaurant reviews |
| Model Selection | Picking the right LLM for the job | Swiggy using small models for autocomplete |
| Prompt Engineering | Writing instructions the model follows | ChatGPT's system prompt making it polite |
| RAG | Giving the model access to your data before answering | Bank chatbot retrieving your account balance |
| Fine-tuning | Training the model on your specific domain | Hospital model trained on medical terminology |
| Agents | Models that can take actions and use tools | Cursor reading, writing, and running code |
| Deploy | Making it available to real users via API | ChatGPT's web interface |
| Evaluate | Measuring if the AI actually works well | Testing accuracy, hallucination rate |

## 5. Live Demo Recap

What we saw in the Jupyter notebook:

- **Tokenization:** Used tiktoken to break text into tokens. Hindi text uses more tokens than English.
- **LLM API call:** Called a local Qwen model from Python using the Ollama API.
- **System prompts:** Same question with different system prompts gave completely different responses (helpful assistant vs pirate vs professor).
- **Temperature:** Temperature 0.0 = same answer every time. Temperature 1.0 = different answer every time. Low = safe. High = creative.
- **Streaming:** Watched tokens appear one by one, just like ChatGPT's typing effect.

### 6. HuggingFace

- HuggingFace (huggingface.co) is the GitHub of AI
- Over 500,000 models available for free
- Model cards show size, license, benchmarks, and usage instructions
- We will use it throughout the course

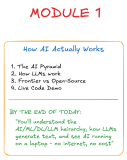
---

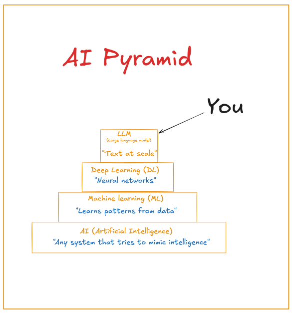
## Key Takeaways

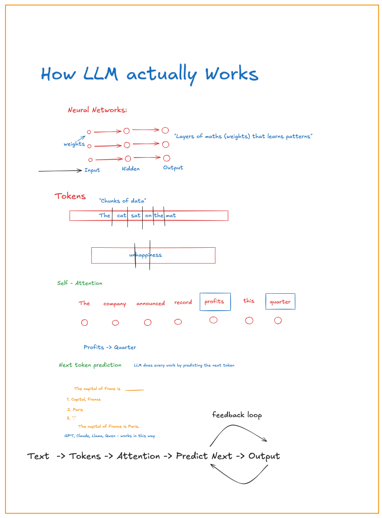
1. **AI, ML, DL, LLM** form a pyramid. Each builds on the one below. You work at the top.
2. **LLMs predict the next token**, one at a time. That is the entire mechanism.
3. **Open-source models are free**, private, and customizable. Closed models are powerful but come with cost and data trade-offs.
4. **The GenAI lifecycle** has 8 stages from data to evaluation. Each stage is a career path. This course covers all of them.
5. **You can run AI on your own laptop** with Ollama. No internet, no API cost.

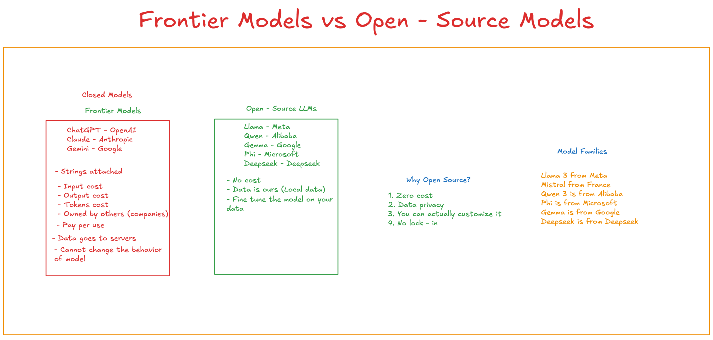
---

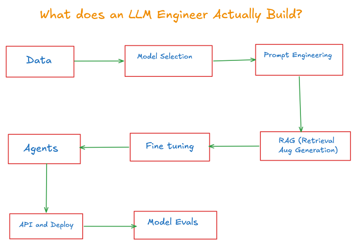
## Session Whiteboard




---

\newpage

# Setup Guide

Everything you need to install before Session 4 (Sunday).

Instructions are provided for Mac (Apple Silicon), Mac (Intel), Windows, and Linux.

---

## Required for Session 4

### 1. Python 3.10 or higher

**Mac (Apple Silicon and Intel):**

macOS comes with an older Python. Install the latest version.

Option A - Download the installer:

1. Go to https://www.python.org/downloads/
2. Download the latest version (3.12 or 3.13)
3. Run the installer

Option B - Using Homebrew (if you have it):

```
brew install python
```

After installing, open Terminal and verify:

```
python3 --version
```

On Mac, the command is `python3`, not `python`.

**Windows:**

1. Go to https://www.python.org/downloads/
2. Download the latest version (3.12 or 3.13)
3. Run the installer
4. **Important:** Check the box that says "Add Python to PATH" before clicking Install

After installing, open Command Prompt or PowerShell and verify:

```
python --version
```

If `python` does not work, try `python3` or `py`.

**Linux (Ubuntu/Debian):**

```
sudo apt update
sudo apt install python3 python3-pip python3-venv
python3 --version
```

**Linux (Fedora):**

```
sudo dnf install python3 python3-pip
python3 --version
```

---

## 2. Ollama

Download from https://ollama.com/download

**Mac (Apple Silicon - M1/M2/M3/M4):**

1. Download the Mac installer from the link above
2. Open the downloaded file and drag Ollama to Applications
3. Open Ollama from Applications (it runs in the menu bar)
4. Open Terminal and run: `ollama pull qwen3:0.6b`

Apple Silicon Macs run Ollama very well. The model uses your GPU automatically.

**Mac (Intel):**

1. Download the Mac installer from the link above
2. Open the downloaded file and drag Ollama to Applications
3. Open Ollama from Applications
4. Open Terminal and run: `ollama pull qwen3:0.6b`

Intel Macs will run the model on CPU. It will be slower than Apple Silicon but it works fine for our course. If your Intel Mac has less than 8 GB RAM, consider using Google Colab as a backup.

**Windows:**

1. Download the Windows installer from the link above
2. Run the installer and follow the steps
3. Open Command Prompt or PowerShell and run: `ollama pull qwen3:0.6b`

If you have an NVIDIA GPU, Ollama will use it automatically. If not, it runs on CPU which is slower but works.

**Linux:**

```
curl -fsSL https://ollama.com/install.sh | sh
ollama pull qwen3:0.6b
```

**Test it (all platforms):**

```
ollama run qwen3:0.6b
```

Ask it anything. If it responds, you are ready. Type `/bye` to exit.

This model is about 500 MB. On a slow connection, the download may take 10-15 minutes.

---

## 3. Code Editor

Install any code editor you prefer:

- **VS Code** (recommended): https://code.visualstudio.com/

- **Cursor**: https://cursor.com/
- Any other editor you are comfortable with

### 4. Jupyter Extension (inside your editor)

If you are using VS Code or Cursor:

1. Open the editor
2. Go to Extensions (left sidebar)
3. Search for "Jupyter"
4. Click Install

This lets you run notebook files (.ipynb) which we will use for all coding sessions.

---

## Install This Week (not needed for tomorrow)

### 5. Git

**Mac:** Git usually comes pre-installed. Run `git --version` to check. If not installed, macOS will prompt you to install Command Line Tools.

**Windows:** Download from https://git-scm.com/downloads and run the installer with default settings.

**Linux:** `sudo apt install git` (Ubuntu/Debian) or `sudo dnf install git` (Fedora).

### 6. Google Colab (just bookmark it)

Go to https://colab.research.google.com and sign in with your Google account.

Use Colab as your backup if your laptop has less than 4 GB RAM, struggles with Ollama, or you cannot install software on your machine.

---

## Verify Your Setup

**Mac/Linux:**

```
python3 --version
ollama --version
git --version
```

**Windows:**

```
python --version
ollama --version
git --version
```

Then test the model: `ollama run qwen3:0.6b`

Ask it a question. If it answers, you are ready for Session 4.

---

## Troubleshooting

**Python not found (Mac):** Try `python3` instead of `python`. Close and reopen Terminal after installation.

**Python not found (Windows):** Reinstall Python with "Add Python to PATH" checked. Try `py` or `python3`.

**Ollama not found (Mac):** Open the Ollama app from Applications first. Reopen Terminal.

**Ollama not found (Windows):** Restart your computer. Check the system tray (bottom right).

**Ollama download stuck:** Press Ctrl+C and try again: `ollama pull qwen3:0.6b`

**Old laptop / less than 4 GB RAM:** Use Google Colab as your backup: https://colab.research.google.com

**Other issues:** Reach out to our support team and we will help you sort it out.

---

## What We Will Set Up Together in Session 4

- Virtual environments (venv / conda) for keeping your projects clean
- Project folder structure for AI projects
- Calling Ollama from Python code

You do not need to set these up beforehand. We will walk through them step by step in class.